Algorithm Design and Analysis

Russell Impagliazzo
russell@cs.ucsd.edu

Lect: Amortized analysis of Data Structures

Thanks, Miles Jones

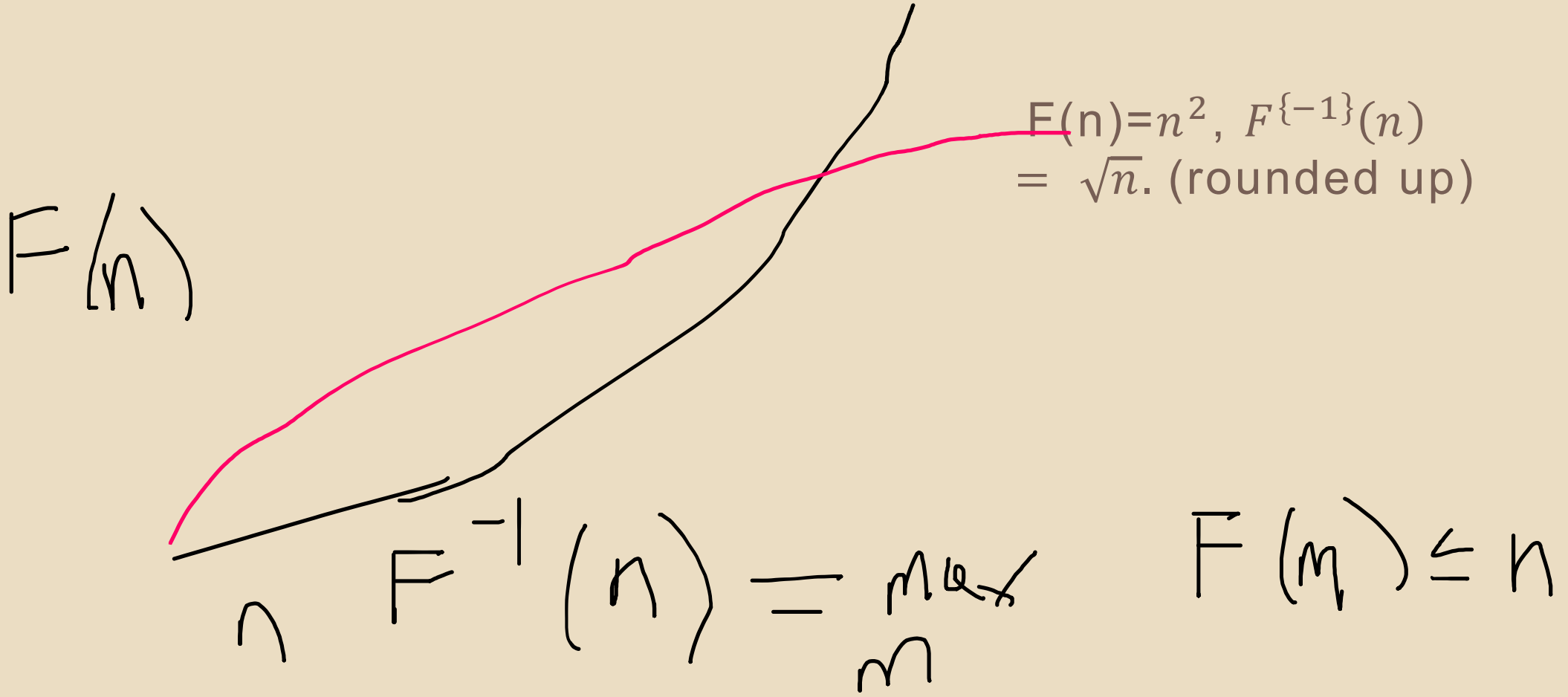
CSE 202

FUNCTIONS AND INVERSE FUNCTIONS

$Let\ F(n)$ be an increasing function from positive integers to positive integers.

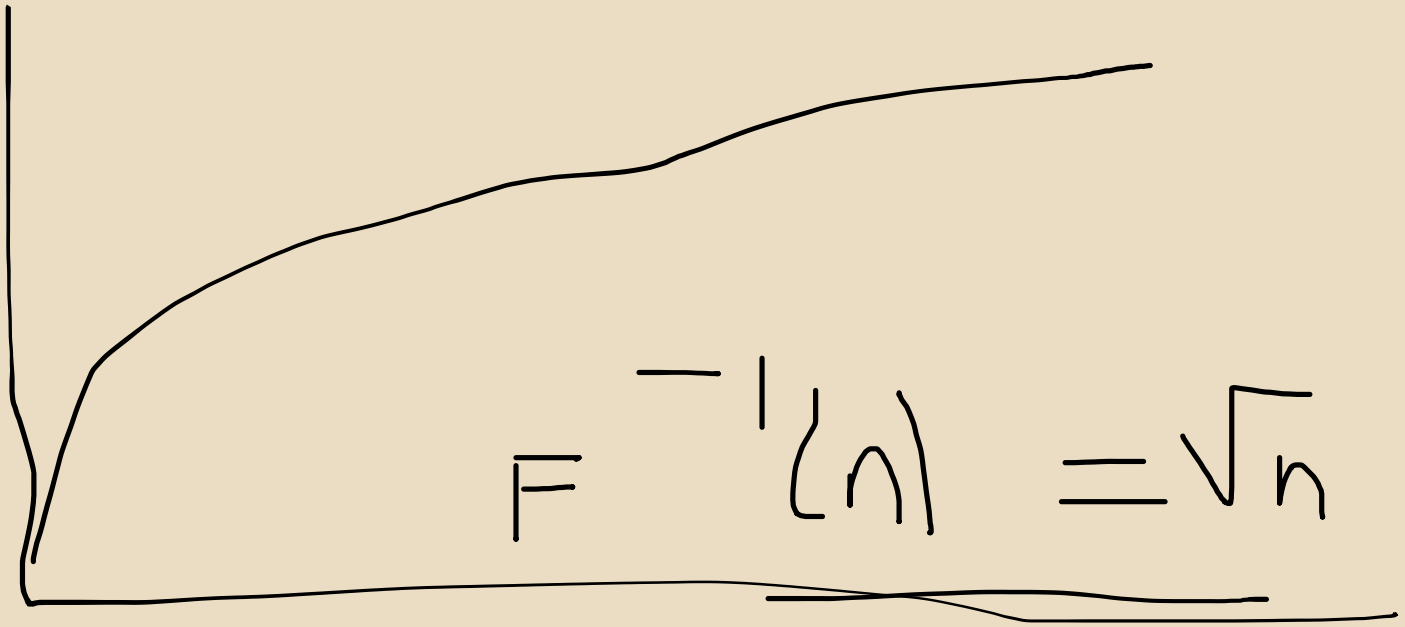
Define the inverse function, $F^{\{-1\}}(n) = \min_{m, F(m) \geq n} m$

WHAT ARE SOME MODERATE SIZED FUNCTIONS?



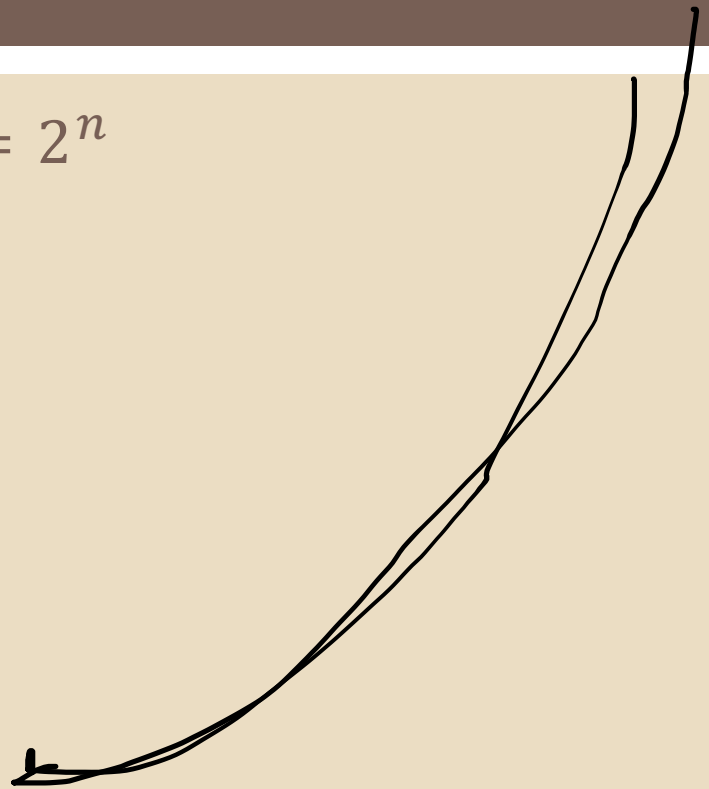
MODERATE SIZED FUNCTIONS

The inverse of a moderately growing function is a moderately growing function.



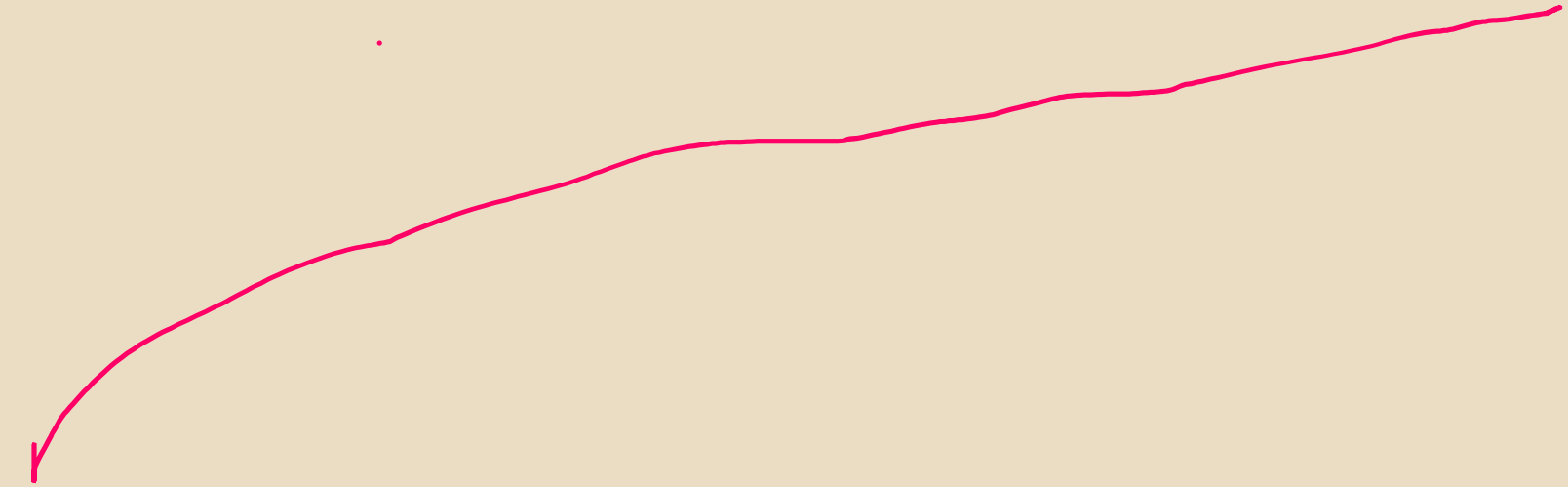
LARGE FUNCTIONS

$F(n) = 2^n$



INVERSES OF LARGE FUNCTIONS

The inverse of a fast growing function is a slowly growing function

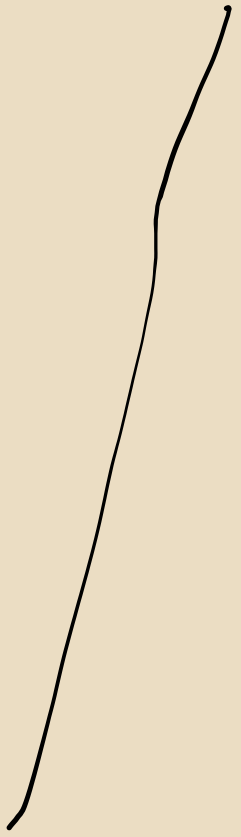


$$F^{-1}(n) = \log n$$

SUPER LARGE FUNCTIONS

DE(n)= 2^{2^n},

TE(n)= $2^{2^{2^n}}$



INVERSES OF SUPER-LARGE FUNCTIONS

DE(m)=n

2^{2^m} =n

$2^m = \log n$

m = log (log n)

$DE^{\{-1\}}(n) = \log(\log(n))$

$TE^{\{-1\}}(n) = \log(\log(\log(n)))$



SIZE OF UNIVERSE

``In astrophysics, the **Eddington number**, N_{Edd}, is the number of protons in the observable universe. Eddington originally calculated it as about 1.57×10^{79}; current estimates make it approximately 10^{80}.[1]''
wikipedia

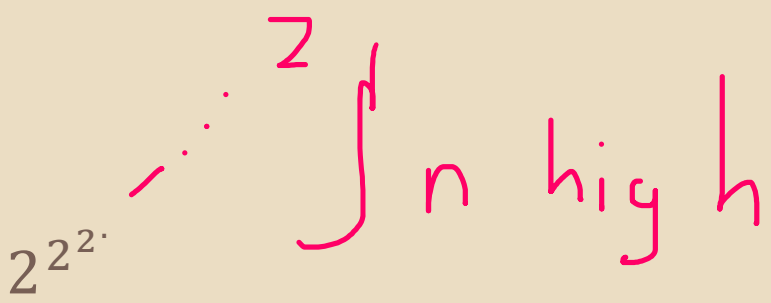
$\log(10^{81}) \approx 270$

$log\ (270) < 9$

$log\ (9) < 4$

TOWER FUNCTION

T(n)=



T(1)=2, T(n)= $2^{T(n-1)}$

INVERSE TOWER

The inverse of the tower function is called log-star

$$\log^* n = \log^*(\log(n)) + 1,\ \log^* 2 = 1$$

We took log of the Eddington number 4 times to get to 2, so

$$\log^*(10^{80}) = 5$$

KEEP GOING

Define a sequence of functions

$$F_0(n) = 2n$$

$$F_k(1) = 2,\ F_k(n) = F_{k-1}\big(F_k(n-1)\big).$$

$$F_1(n) = 2^n$$

$$F_2(n) = Tower(n)$$

$$F_3(n)\ is\ a\ tower\ of\ towers\ of\ towers$$

$$\beta_k\ is\ the\ inverse\ of\ F_k$$

KRUSKAL'S ALGORITHM USING DSDS

- procedure kruskal(G,w)
  - input: undirected graph G with edge weights w
  - output: A set of edges X that defines a minimum spanning tree
- for all $u \in V$
  - makeset(u)
- $X = \{\}$
- sort the edges E in increasing order by weight
- for all edges $(u, v) \in E$
  - if find(u) ≠find(v):
    - add edge (u, v) to X
    - union(u, v)

DISJOINT SET DATA STRUCTURE OPERATIONS

- makeset (u): This creates a set with one element, u
- find(u): finds the set to which u belongs
- union(u,v): merges the sets containing u and v

Runtime of Kruskal's:

$$|V|makeset + 2|E|find + (|V| - 1)union + sort(|E|)$$

DSDS USING TREES

- Each set is a rooted tree, with the vertices of the tree labelled with the elements of the set and the root the leader of the set
- Only need to go up to leader, so just need parent pointer, not children pointers
- Thus, we don’t need to restrict the degrees or fan-out of the tree

A DATA STRUCTURE FOR KRUSKAL'S (DIRECTED TREES WITH RANKS)

Each set is a tree

The root of the tree is the leader of the set

Each vertex has a parent pointer, p(x), and the root is defined so that p(root)=root

In addition, each vertex will have a field rank(x) . Rank(x) intuitively represents the depth of the tree at x, but we might later decrease the depth Without changing rank(x), so in general, it is just an upper bound on the depth.

DSDS OPERATIONS

MakeSet (x): p(x)=x, rank(x)=0

Merge(x,y): x, y, roots : Make larger rank parent of smaller rank. If ranks are equal, increment new parent's rank.

ONE CHANGE: PATH COMPRESSION

Find (x).

If x = p(x) return x

p(x):=Find(p(x))

Return p(x)

``Store and reuse, don't recompute''

Note that this changes the pointers all the way up the path, to be pointing directly to the root

Time= O(length of path from x to root +1)

INVARIANT: RANKS INCREASE

Lemma: For every v that is not a root, rank(p(v)) > rank(v)

Invariant: Vacuously true at start, since every v is root

When a vertex v becomes the child of some other vertex u, it is because u and v's trees were just merged, and v had smaller rank. If they tied, Rank (u) was incremented, so either way, the inequality is true after the merge..

The rank of v stays the same afterwards. Since the parent of v only moves up the tree, the rank of the parent only increases afterwards (using invariant in induction argument)

INVARIANT: SIZE IS EXPONENTIAL IN RANK

Lemma: If x is a root, then the tree at x has at least $2^{\{rank(x)\}}$ elements

True at start, since rank(x)=0, size=1

Find doesn't change root, rank or size of tree

Merge: If we are merging u, v, and rank(u)>rank(v), rank(u) doesn't change, but the size of the tree only grows.

If they are both rank r before the merge, both trees are size at least 2^r before the merge and the parent is rank r+1 after the merge. The total Size is at least $2 * 2^r = 2^{\{r+1\}}$, so the invariant still holds.

HEIGHT IS LOGARITHMIC

Since |V| $\geq size\ of\ tree \geq 2^{\{rank(root)\}}$, rank root $\leq \log n$

Since ranks along paths increase, depth is at most rank(root).

So the height of any path up the tree is at most log n

Thus, the time for Find is O(log n), and merge is O(1),

Total time for Kruskals: O(m log n + n + m log n) = O(m log n)

WORST CASE TIMES

Find: O(log n)
Merge : O(1) + 2 finds

Upper bound on Kruskal's: n Merges + m Finds = O((n+m) log n)

But is this tight?

THEME

Doing something many times can often be faster than doing it Independently many times

AMORTIZED ANALYSIS

Amortized analysis: Bound total time of m operations, rather than worst-case time for single operation * m

Intuition: Fast operations make things worse in the future, but slow operations make things better in the future. (Merging might build up the height of the tree, but finding a deep vertex shrinks the average heights.).

Potential function method: For the proof, we define some potential function P_t: Some measure of how bad the situation is after t'th operation. We need $P_0 = 0, P_t \geq 0$.

TOY EXAMPLE

Coffee drink costs 2 dollars.

But you also get a punch on a card, and if you fill a card with 7 punches you get a free coffee.

How much does coffee really cost? How much is a punch worth?

Potential function: p* #Punches.

Purchase directly : Cost: 2. Amortized cost: 2-p

Trade in our card: Cost: 0 Amortized cost: 7p

To see correct for p, 2-p=7p , p = .25 , amortized cost of coffee: 1.75

8 cups of coffee costs 14 , 14/8= 1.75

AMORTIZED COSTS OF OPERATIONS

Define $AT_t = (Time\ for\ operation\ t) + P_t\ - P_{t-1}$

$Summing\ up\ amortized\ times$:

$AT_1 + AT_2 + AT_3 + \cdots AT_m = (Time\ for\ first\ op) + P_1 - P_0$

$+\ (Time\ for\ 2nd\ op) + P_2 - P_1$

$+\ (Time\ for\ 3rd\ op) + P_3 - P_2$

+.....+

$+\ (Time\ for\ m'th\ op) + P_m - P_{m-1} =$

Time for all ops $+P_m$. $-P_0$

$\geq Time\ for\ all\ ops$

AMORTIZED TIME BOUNDS TOTAL TIME

Thus, the total amortized time is an upper bound on the total time of all operations.

Then the total time for m operations is at most m* Worst-case amortized time of an operation

While amortized time is an ``average time'' for an operation, we are still doing a worst-case analysis of the entire algorithm, because we are averaging over the operations, not over inputs.

SIMPLE EXAMPLE: CASH REGISTER

Bills have powers of 10 denominations, 1, 10, 100, 1000, 10000

When we reach 10 of one denomination, trade in for larger.

Bills deposited one at a time.

If m bills are deposited, and the register starts empty, bound the total number of trades.

Potential = 1/9 (total number of bills we have).

WORST-CASE FOR ONE DEPOSIT

If we have n consecutive denominations each with 9 bills, one deposit could cause n trades.

But that situation needs to be built towards. Time = number of swaps

P_t=. 1/9 (Total number of bills in register at time t.)

AT_t = 1+(Trades for deposit t) + 1/9(change in number of bills in register for that deposit)

If we make no trades, AT_t = 1 1/9, for increasing bills.

Each trade actually costs 0 amortized time. One trade, but replace 10 bills with 1=-1 potential function. Thus, $AT_t = 1\ 1/9$ no matter what happens.

Thus, if we start with empty register, total number of trades is at most 10/9 * the number of deposits.

TOKENS

To do a similar analysis for union-find, imagine giving out coupons to the elements. The larger the rank, the more coupons the element will have. Each coupon will be worth ``one free pointer change'' for a future operation.

We'll give rules in terms of the algorithm for handing out and using coupons, but these are just for the analysis; the algorithm itself does not change.

RULES FOR TOKENS

Make set: Give the new element 1 token

Merge: Give the new root one token

Find: If a vertex's pointer changes (i.e., it is not the leader, and its parent is not the leader), and it has at least one token, it spends that token

P_t = C * Total unspent tokens, where C is the constant in the cost of Find, i.e., where time to find x = C* (length of path from x to root)

AMORTIZED COSTS OF OPERATIONS

Make-set : O(1)+ Token = O(1)

Merge: O(1) + Token = O(1)

Find: O(1)+ C* (number of pointer changes along path) -C* (number of vertices whose pointers change who have at least one token) = O(1)+ O(number of ``broke'' vertices along path)

Here, a vertex is broke if it has no more tokens to spend

BROKE VERTICES

Claim: When a vertex v goes broke, rank(p(v)) $> 2 * rank(v)$

Proof: each time a vertex spends a token, its parent is replaced by an ancestor of the parent, which has strictly greater rank. To go broke , this has to happen its number of tokens times, which started at rank(v) . So rank(p(v)) started at rank(v)+1 and increased rank(v) times

NUMBER OF BROKE VERTICES ALONG PATH

Claim: When a vertex v goes broke, rank(p(v)) $> 2 * rank(x)$

Corollary: There are at most log (log (|V|))+1 broke vertices along any path.

Proof: say $v_1, v_2, .. v_k$ are broke and along a path.

$\log |V| \geq rank\big(p(v_k)\big) \geq 2rank(v_k) \geq 2rank\big(p(v_{k-1})\big) \geq 4rank(v_{k-1})$
$\geq 4\, rank\big(p(v_{k-2})\big) \geq 8rank(v_{k-2}) \geq \cdots \geq 2^{k-1} rank(v_2) \geq 2^{k-1}$

AMORTIZED TIMES

Merge: O(1)+C = O(1)

Makeset : O(1)+C = O(1)

Find : maximum number of broke vertices along path+O(1) = O(log log |V|)

TIME ANALYSIS OF KRUSKAL

n make set operations: O(n)

n merge operations: O(n)

m find operations: $O(m \log \log n)$

In cases when sorting is linear time, this last is an upper bound on total Kruskal time

BUT IS THIS TIGHT?

This is really pretty fast.

But is this the actual time for DSDS?

Actually no. We can give a similar but better analysis, using more complex token rules.

Introduce two types of tokens, self tokens and ``gift tokens'' that will only be used on behalf of the vertex's parent. Potential is C(# of unspent self tokens + unspent gifts)

RULES FOR TOKENS AND GIFTS

Make set: Give the new element a self token and a gift token

Merge: Give the new root a token and a gift token
Leaders always have more than their rank in tokens and in gifts

Find: Let x be any element along the path. Let x's current parent be y. If y's parent changes and y has no tokens, and x has an unspent gift, x spends a gift token. If x's parent changes and x has a token, x spends a self token.

AMORTIZED COSTS OF OPERATIONS

Make-set : O(1)+ 2*Token = O(1)

Merge: O(1) + 2*Token = O(1)

Find: O(1)+ C* (number of pointer changes along path) -Token* (number of elements whose pointers change and who have at least one self token or whose child on the path has an unspent gift token) = O(1)+ O(number of ``unpaid'' elements along path) $\leq O(1 + number\ of\ elements\ on\ path\ with\ no\ unspent\ gift\ tokens)$.

(The constants are for the leader, the child of the leader, and the element we're looking for, which has no child on the path to pay with a gift token if it is broke.)

WHAT HAPPENS WHEN AN ELEMENT SPENDS A GIFT?

Lemma: After an element spends a gift, the rank of its parent more than doubles.

If x spends a gift , its parent y has no tokens and its parent's pointer changes. x's pointer changes to the same value as y's, the new leader. Since y is broke, rank(p(y)) $> 2\ rank(y)$. x's old parent was y, and x's new parent will be the set leader, which is an ancestor of p(y) and hence of higher rank than p(y). So the new rank of p(x) will be greater than 2 rank(y), more than twice the former rank of p(x).

WHEN IS AN ELEMENT OUT OF GIFTS?

x had more than rank(x) gifts to start, and rank(p(x))> rank(x) at the start (i.e., the first time x is not a set leader). So rank(p(x))≥ 1 at the start.

Every time x spends a gift, rank(p(x)) more than doubles. So after spending j gifts, rank(p(x)) ≥ 2^j. After spending all more than rank(x) gifts, $rank(p(x)) \geq 2^{rank(x)}$.

HOW MANY UNPAID FOR ELEMENTS ALONG PATH

Let $y_1, y_2, \ldots y_k$ be the unpaid for vertices along the path, from bottom to top. y_1 might be the first element, but the others have children on the path, $x_2, \ldots x_k$. Since $x_2, \ldots x_k$ have no gift certificates, $rank(y_i) \geq 2^{rank(x_i)}$ for all i.

Then $rank(y_k) > 2^{rank(x_k)} \geq 2^{rank(y_{k-1})} > 2^{2^{rank(x_{k-1})}} \geq 2^{2^{rank(y_{k-2})}} \geq \cdots$

$2^{2^{\cdot^{\cdot^{\cdot^{2^{rank(x_2)}}}}}}$

$\geq$ Tower $(k-2)$

AMORTIZED COST OF FIND

Since $\log n \geq rank(y_k) \geq Tower(k-2)$, taking the inverse function gives. $k \leq \log^*(\log n) + 2 = \log^* n$ + 1

So the amortized cost of find is at most $O(\log^* n)$

and the non-sorting time for Kruskal is O(|E| $\log^* |V|$)

KEEPING ON GOING

To change the analysis to the inverse of the fifth Ackerman function, add ``gift certificates for grandparents'' . For the sixth, add ``gifts for great-grandparents''.

Each adds a factor to the cost of Make set, Merge

Break-even point= inverse Ackerman

MANY TYPES OF TOKENS

Imagine giving out K (which will be a function of n) different types of tokens to the elements. The larger the rank, the more coupons the element will have. Each coupon will be worth ``one free pointer change'' for a future operation.

Type 0 can be spent on yourself, type 1 on your parent, type 2 on your grandparent, type 3 on your great-grandparent, and so on, with type i tokens only spent on behalf of your i'th generation ancestor.

We'll give rules in terms of the algorithm for handing out and using coupons, but these are just for the analysis; the algorithm itself does not change.

RULES FOR TOKENS

Make set: Give the new element 1 token of each type, 0 to K-1.

Merge: Give the new root one token of each type, 0 to K-1.

Find: For all $0 \leq i < K$, if the element's i'th generation ancestor's pointer will change (i.e., it is not the leader, and its parent is not the leader), and none of your ancestors (including that one) have spent a token on it's behalf, and you have a token of type i, spend a token of type i.

P_t = C * Total unspent tokens, where C is the constant in the cost of Find, i.e., where time to find x = C* (length of path from x to root)

AMORTIZED COSTS OF OPERATIONS

Make-set : O(1)+ K Token = O(K)

Merge: O(1) + K Tokens = O(K)

Find: O(1)+ C* (number of pointer changes along path) -C* (number of vertices where they or one of their descendents spend a token on their behalf) = O(1)+ O(number of ``unpaid'' pointer changes)

So we need to bound the last number.

ACKERMAN HIERARCHY

Remember the Ackerman hierarchy is defined by

$$F_0(n) = 2n$$

$F_i(1) = 2, F_i(n+1) = F_{i-1}(F_i(n))$;

i.e., do F_{i-1} n times starting from 2

(e.g., $F_1(n) = 2^n, F_2(n) = Tower(n)$)

$Ack(n) = F_n(n)$. $\alpha(n) = Ack^{-1}(n)$

RANKS OF PARENTS INCREASE SUPERFAST

Lemma: If x has spent j tokens of type i, then $rank\big(p(x)\big) \geq F_i(j)$

If x has no tokens of type i left , then $rank(p(x)) \geq F_i(rank(x))$

We'll prove these statements by intermeshed induction, first on i, then on j. The top claim for i+1 will follow from the bottom for i, and the bottom for i+1 from the top for i+1.

RANKS OF PARENTS INCREASE SUPERFAST

Lemma: If x has spent j tokens of type i, then $rank\big(p(x)\big) \geq F_i(j)$

If x has no tokens of type i left , then $rank(p(x)) \geq F_i(rank(x))$

Base case: i=0, second part was already shown.

Assume the second claim is true for type i tokens. When we spend the first token of type i+1, our i+1'st generation ancestor's pointer changes, so our new rank is at least i+2 > 2 =$F_{i+1}(1)$.

RANKS OF PARENTS INCREASE SUPERFAST

Lemma: If x has spent j tokens of type i, then $rank\big(p(x)\big) \geq F_i(j)$

If x has no tokens of type i left , then $rank(p(x)) \geq F_i(rank(x))$

Assume the second claim is true for type i tokens. Assume after x has spent j tokens of type i+1 , rank(p(x)) $\geq F_i(j)$. When x spends the next token, let y be its parent, and r the root. If y had a token of type i, it would have spent it on behalf of its i'th ancestor, which is x's i+1'st ancestor. So x wouldn't spend the token. By the IH, rank(r)$\geq rank\big(p(y)\big) \geq F_i\big(rank(y)\big) = F_i(rank\big(p(x)\big) \geq F_i\big(F_{i+1}(j)\big) = F_{i+1}(j + 1)$

RANKS OF PARENTS INCREASE SUPERFAST

Lemma: If x has spent j tokens of type i, then $rank\big(p(x)\big) \geq F_i(j)$

If x has no tokens of type i left , then $rank(p(x)) \geq F_i(rank(x))$

Assume the first claim is true for type i+1 tokens. Since x has at least rank(x) tokens of type i+1 before it spends any, x has spent at least j=rank(x) tokens of type i+1. Therefore, , rank$(p(x)) \geq F_{i+1}(rank(x))$

NUMBER OF UNPAID POINTER CHANGES ALONG PATH

Claim: There are at most K+ $F_{K+1}^{-1}(\log n)$ unpaid pointer changes along any path.

Say $v_1, v_2, .. v_L$ were all unpaid. Except for the first K,

the K'th descendents $x_{K+1}, ...x_L$. are along the path. And none of them had tokens of type K left.

$rank(x_{K+1}) \geq 2 = F_{K+1}(1)$,

$rank(x_{K+2}) \geq rank(p(x_{K+1})) \geq F_K\big(rank(x_{K+1})\big) \geq F_{K+1}(2)$

$.. rank(x_{K+(L-K)}) \geq rank(p(x_{K+(L-K-1)})) \geq F_K\left(rank\left(x_{K+(L-K-1)}\right)\right) \geq$

$F_{K+1}(L-K-1) = F_{K+1}(L-K)$

log n $\geq F_{K+1}(\mathrm{L}-\mathrm{K})$

AMORTIZED TIMES

Merge: O(1)+C*K = O(K)

Makeset, per element : O(K)

Find : O($K + F_K^{-1}(n)$).

Balance: Pick K so that K=$F_K^{-1}(n),\ i.e., F_K(K) = n, K = \alpha(n)$

All costs are $O(\alpha(n))$

TOTAL TIME

If we can sort in linear time, and we use this data structure, time for Kruskal's algorithm is

O((m+n) $\alpha(n)$) where α is the inverse of the Ackerman function.

IS THIS TIGHT ?

Tarjan: Union-find can take amortized time at least inverse Ackerman

Fredman-Saks: This is true for any implementation of DSDS in the very general cell probe model.

So although the Ackerman function doesn't seem to have anything to do with this simple data structure, this really is the time for DSDS.

GREEDY APPROXIMATION ALGORITHMS

Sometimes, we still want to use greedy algorithms (or other algorithms) when they do not give optimal solutions

Maybe finding optimal solutions is NP-complete, or the greedy algorithm is much, much faster than an exactly optimal algorithm for the problem

We can sometimes give approximation guarantees:

Cost(GS) ≤ C Cost(OS)

Value(GS) ≥ Value(OS)/C

C is called approximation ratio

Next class: bounding approximation ratios of greedy algorithms